EDA

Cooper Ruffing

```
library(readxl)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.1.6
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(dplyr)
library(broom)
library(MASS)
```

```
Attaching package: 'MASS'

The following object is masked from 'package:dplyr':

    select
```

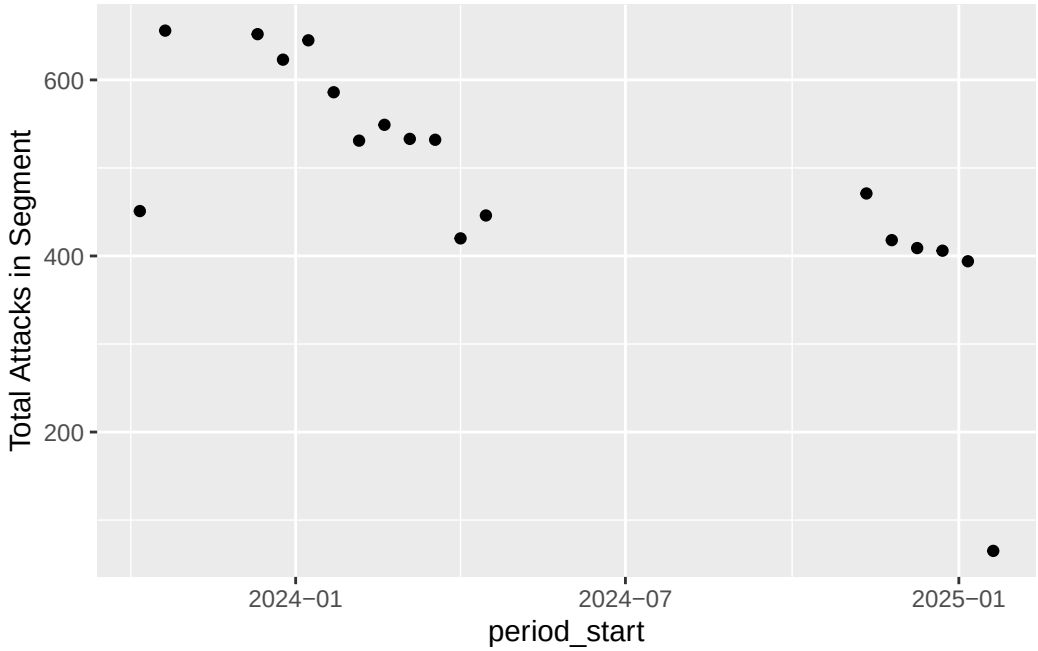
```
library(AER)
```

```
Loading required package: car
Loading required package: carData
```

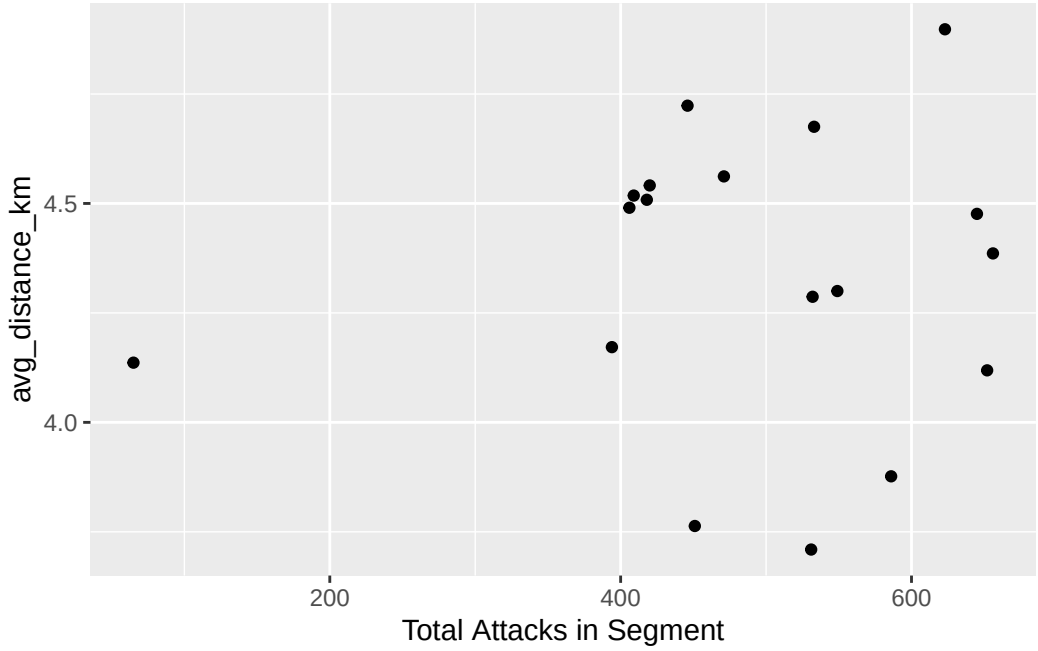
```
Attaching package: 'car'

The following object is masked from 'package:dplyr':

    recode

The following object is masked from 'package:purrr':

    some

Loading required package: lmtest
Loading required package: zoo

Attaching package: 'zoo'

The following objects are masked from 'package:base':

    as.Date, as.Date.numeric

Loading required package: sandwich
Loading required package: survival
```

```r
library(car)
library(sandwich)
library(lmtest)

totaldata <- read_excel("aggregated_dataset.xlsx")
```

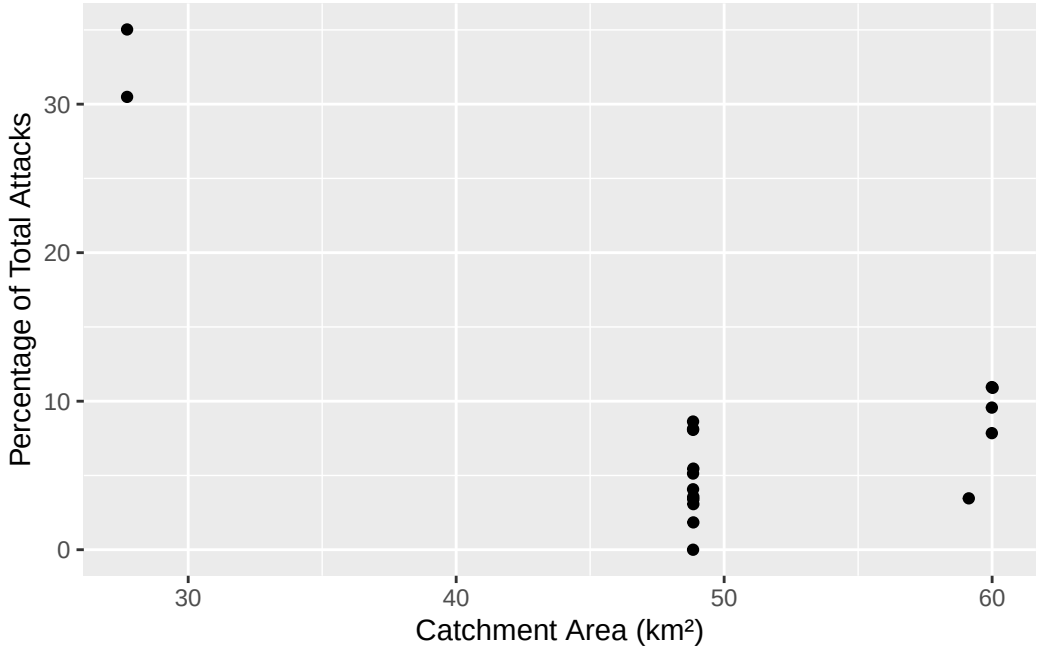
```r
ggplot(totaldata, aes(x = period_start, y = `Total Attacks in Segment`)) +
  geom_point()
```



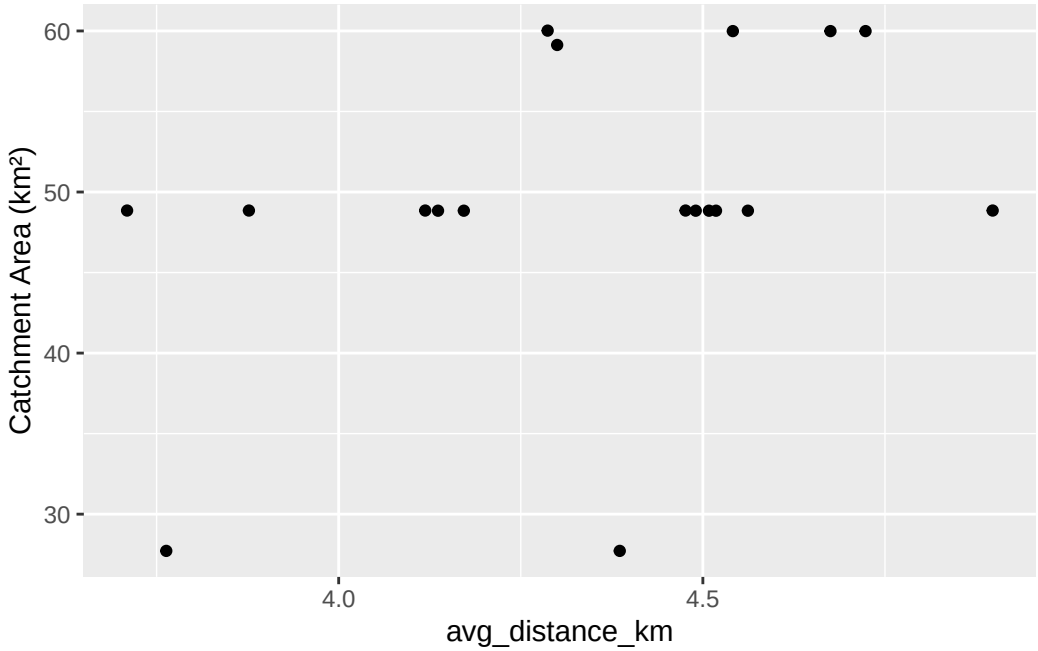

```
ggplot(totaldata, aes(x = `Total Attacks in Segment`, y = avg_distance_km)) +
  geom_point()
```



```
ggplot(totaldata, aes(x = `Catchment Area (km²)`, y = `Percentage of Total Attacks`)) +
  geom_point()
```




```
ggplot(totaldata, aes(x = avg_distance_km, y = `Catchment Area (km²)`)) +
  geom_point()
```




```
totaldata <- totaldata |>
  mutate(totinjuries = Burn + CMF + Limb + NTD + `Soft Tissue` + `Wound Care`) |>
  mutate(burnprop = Burn / totinjuries) |>
  mutate(cmfprop = CMF / totinjuries) |>
  mutate(limbprop = Limb / totinjuries) |>
  mutate(ntdprop = NTD / totinjuries) |>
  mutate(stprop = `Soft Tissue` / totinjuries) |>
  mutate(wcprop = `Wound Care` / totinjuries)

totaldata <- totaldata |>
  mutate(across(where(is.numeric), ~ replace(., is.nan(.), 0)))
```

```
# Single robust chunk: uses exact "period_start to period_end" labels on x-axis.
# All tidy functions are namespace-qualified to avoid masking issues (e.g. MASS::select).
#
# Requirements (install once if needed):
# install.packages(c("readxl","dplyr","tidyr","ggplot2","lubridate","scales","stringr"))

# --- Libraries (not strictly necessary because we qualify most calls, but load for plotting
# It's fine to load them but we won't rely on unqualified function names.
suppressMessages({
  library(ggplot2)
})
```

```r
# --- Read the file (tries working dir then /mnt/data) ---
xlsx_candidates <- c("aggregated_dataset.xlsx", "/mnt/data/aggregated_dataset.xlsx")
dataset_path <- NULL
for (p in xlsx_candidates) {
  if (file.exists(p)) { dataset_path <- p; break }
}
if (is.null(dataset_path)) stop("Could not find aggregated_dataset.xlsx. Place it in the work

df <- readxl::read_xlsx(dataset_path)

# --- Normalize column names slightly (trim, collapse whitespace) ---
orig_names <- names(df)
clean_names <- stringr::str_trim(orig_names)
clean_names <- stringr::str_replace_all(clean_names, "[\\s]+", " ")
names(df) <- clean_names
lc_names <- tolower(clean_names)

# --- Find biweekly start/end columns (fuzzy) ---
possible_start_names <- c("period_start", "period start", "start_date", "start", "periodstart
possible_end_names   <- c("period_end", "period end", "end_date", "end", "periodend")

find_first_match <- function(cands, lc_names, avail_names) {
  for (c in cands) {
    i <- which(lc_names == tolower(c))
    if (length(i) > 0) return(avail_names[i[1]])
  }
  for (c in cands) {
    i <- which(stringr::str_detect(lc_names, stringr::str_replace_all(tolower(c), "[^a-z0-9]+
    if (length(i) > 0) return(avail_names[i[1]])
  }
  return(NA_character_)
}
start_col <- find_first_match(possible_start_names, lc_names, names(df))
end_col   <- find_first_match(possible_end_names, lc_names, names(df))

if (is.na(start_col) | is.na(end_col)) stop("Could not detect biweekly period start/end colum

# --- Convert start/end to Date (handle POSIX, Date, or character with timezone) ---
to_date_safe <- function(x) {
  if (inherits(x, "Date")) return(x)
  if (inherits(x, "POSIXt")) return(as.Date(x))
  x_chr <- as.character(x)
```

```r
  # drop trailing " UTC" or timezone-like suffix
  x_chr <- stringr::str_replace(x_chr, "\\s*UTC\\s*$", "")
  # try ymd_hms first, then ymd
  parsed_hms <- suppressWarnings(lubridate::ymd_hms(x_chr, quiet = TRUE))
  if (!all(is.na(parsed_hms))) return(as.Date(parsed_hms))
  parsed <- suppressWarnings(lubridate::ymd(x_chr, quiet = TRUE))
  as.Date(parsed)
}

df[[start_col]] <- to_date_safe(df[[start_col]])
df[[end_col]]   <- to_date_safe(df[[end_col]])

# --- Create period label "YYYY-MM-DD to YYYY-MM-DD" and ordered factor by period start ---
df <- dplyr::arrange(df, .data[[start_col]])
df <- dplyr::mutate(df,
                    period_start_date = as.Date(.data[[start_col]]),
                    period_end_date   = as.Date(.data[[end_col]]),
                    period_label = paste0(format(period_start_date, "%Y-%m-%d"), " to ", form
)

# ordered factor so x-axis respects chronological order
df$period_label <- factor(df$period_label, levels = unique(df$period_label))

# --- Define user variable groups (attempt fuzzy matching to actual columns) ---
attack_vars_user <- c("Air/drone strike","Ground Attacks","Shelling/artillery/missile attack"
injury_vars_user <- c("Burn","CMF","Limb","NTD","Soft Tissue","Wound Care")
rest_vars_user   <- c("avg_distance_km","Catchment Area (km²)","Attacks in Catchment","Total

fuzzy_match <- function(user_vec, available_names) {
  avail_lc <- tolower(available_names)
  res <- character(length(user_vec))
  for (i in seq_along(user_vec)) {
    u <- tolower(user_vec[i])
    exact_idx <- which(avail_lc == u)
    if (length(exact_idx) > 0) { res[i] <- available_names[exact_idx[1]]; next }
    contains_idx <- which(stringr::str_detect(avail_lc, stringr::fixed(u)))
    if (length(contains_idx) > 0) { res[i] <- available_names[contains_idx[1]]; next }
    u_norm <- stringr::str_replace_all(u, "[^a-z0-9]+", "")
    norm_idx <- which(stringr::str_replace_all(avail_lc, "[^a-z0-9]+", "") == u_norm)
    if (length(norm_idx) > 0) { res[i] <- available_names[norm_idx[1]]; next }
    partial_idx <- which(stringr::str_detect(avail_lc, stringr::str_replace_all(u, "[^a-z0-9
    if (length(partial_idx) > 0) { res[i] <- available_names[partial_idx[1]]; next }
```

```
    res[i] <- NA_character_
  }
  res
}

attack_cols <- fuzzy_match(attack_vars_user, names(df)); attack_cols <- attack_cols[!is.na(at
injury_cols <- fuzzy_match(injury_vars_user, names(df)); injury_cols <- injury_cols[!is.na(in
rest_cols   <- fuzzy_match(rest_vars_user, names(df));   rest_cols   <- rest_cols[!is.na(rest

# also add any percent/percentage columns to rest_cols (useful)
pct_cols <- names(df)[stringr::str_detect(tolower(names(df)), "percent|percentage")]
rest_cols <- unique(c(rest_cols, pct_cols))

# Diagnostic print
cat("Using file:", dataset_path, "\n")
```

```
Using file: aggregated_dataset.xlsx
```

```
cat("Period start col:", start_col, "end col:", end_col, "\n")
```

```
Period start col: period_start end col: period_end
```

```
cat("Periods (first/last):", as.character(first(df$period_label)), " ... ", as.character(last
```

```
Periods (first/last): 2023-10-07 to 2023-10-20  ...  2025-01-20 to 2025-02-02
```

```
cat("Attack columns found:", if(length(attack_cols)>0) paste(attack_cols, collapse=", ") else
```

```
Attack columns found: Air/drone strike, Ground Attacks, Shelling/artillery/missile attack, Ci
```

```
cat("Injury columns found:", if(length(injury_cols)>0) paste(injury_cols, collapse=", ") else
```

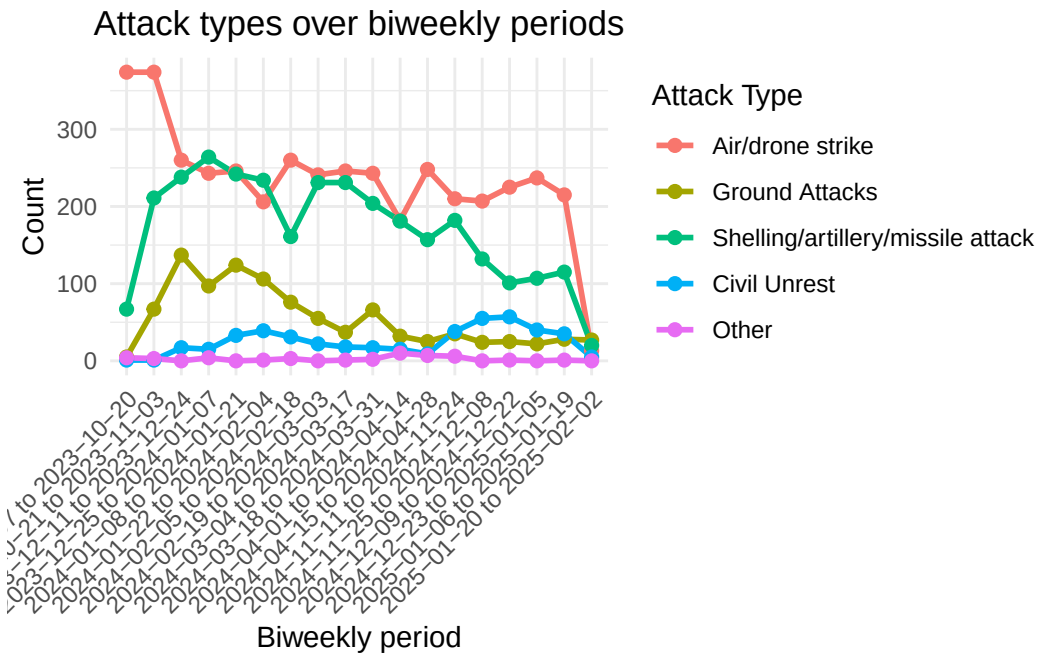
```
Injury columns found: Burn, CMF, Limb, NTD, Soft Tissue, Wound Care
```

```
cat("Rest columns found:", if(length(rest_cols)>0) paste(rest_cols, collapse=", ") else "NONE
```

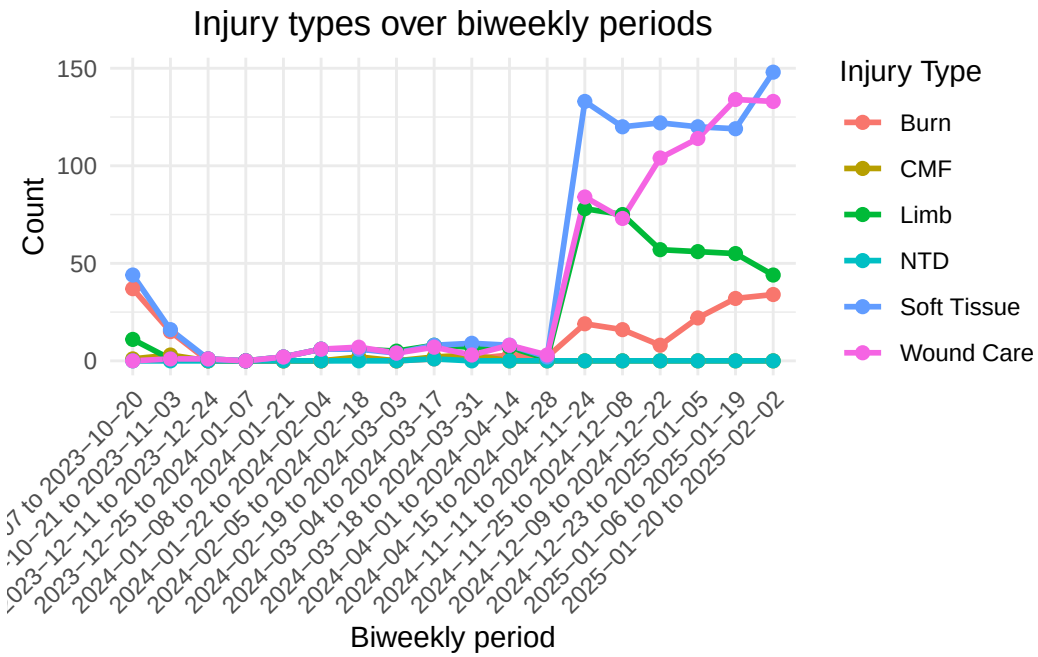
```
Rest columns found: avg_distance_km, Catchment Area (km²), Attacks in Catchment, Total Attack
```

```r
# --- helper numeric coercion ---
to_num <- function(x) suppressWarnings(as.numeric(x))

# --- 1) Attack types plot (x = period_label categorical) ---
if (length(attack_cols) > 0) {
  attacks_long <- df %>%
    dplyr::select(period_label, dplyr::all_of(attack_cols)) %>%
    tidyr::pivot_longer(cols = -period_label, names_to = "attack_type", values_to = "value") 
    dplyr::mutate(value = to_num(value),
                  attack_type = factor(attack_type, levels = unique(attack_type)))

  p_attacks <- ggplot2::ggplot(attacks_long, ggplot2::aes(x=period_label, y=value, group=atta
    ggplot2::geom_line(size=1) +
    ggplot2::geom_point(size=2) +
    ggplot2::labs(title="Attack types over biweekly periods", x="Biweekly period", y="Count"
    ggplot2::theme_minimal() +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 45, hjust = 1), plot.title = g
  print(p_attacks)
} else {
  plot.new(); text(0.5,0.5,"No attack columns found to plot")
}
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```

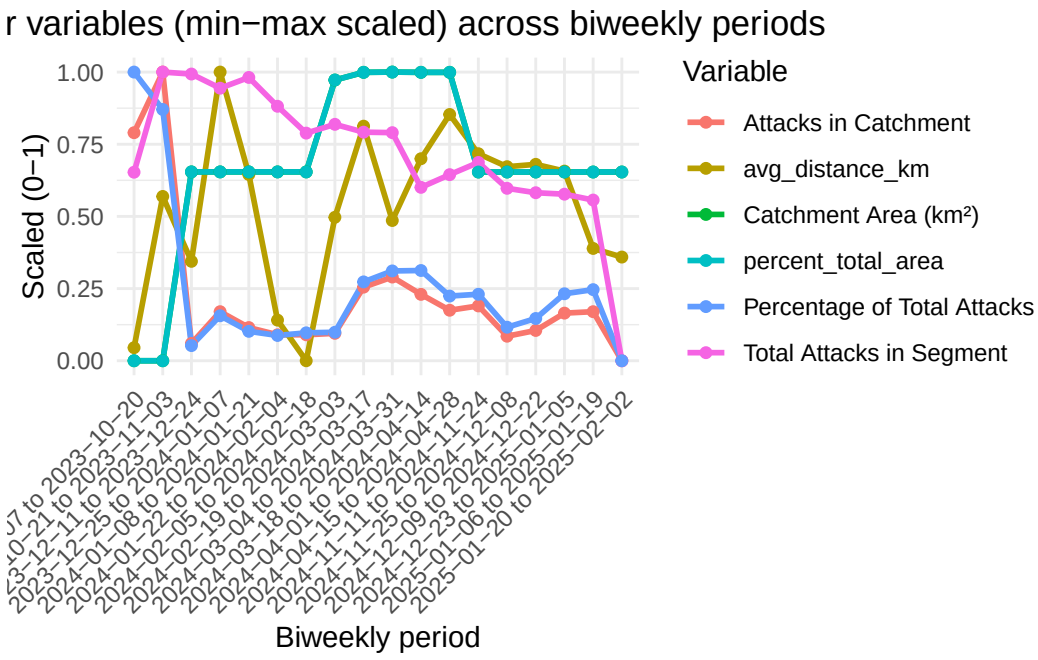


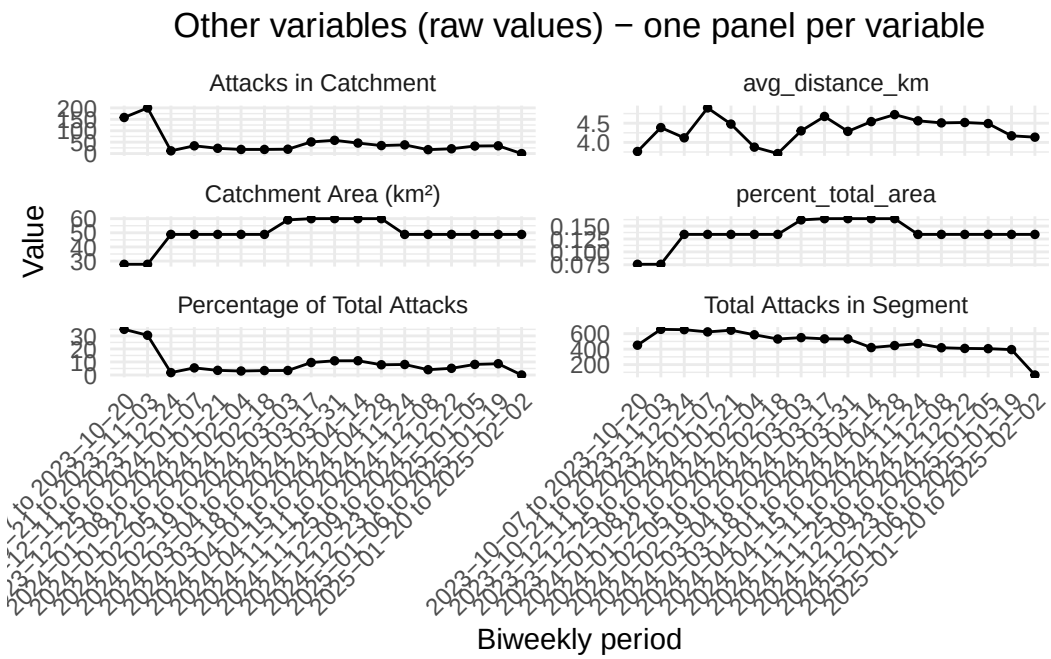

```
# --- 2) Injury types plot ---
if (length(injury_cols) > 0) {
  inj_long <- df %>%
    dplyr::select(period_label, dplyr::all_of(injury_cols)) %>%
    tidyr::pivot_longer(cols = -period_label, names_to = "injury_type", values_to = "value")
    dplyr::mutate(value = to_num(value),
                  injury_type = factor(injury_type, levels = unique(injury_type)))

  p_injuries <- ggplot2::ggplot(inj_long, ggplot2::aes(x=period_label, y=value, group=injury_
    ggplot2::geom_line(size=1) +
    ggplot2::geom_point(size=2) +
    ggplot2::labs(title="Injury types over biweekly periods", x="Biweekly period", y="Count"
    ggplot2::theme_minimal() +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 45, hjust = 1), plot.title = g
  print(p_injuries)
} else {
  plot.new(); text(0.5,0.5,"No injury columns found to plot")
}
```




```
# --- 3) Rest variables combined (min-max scaled) with categorical x ---
if (length(rest_cols) > 0) {
  rest_long <- df %>%
    dplyr::select(period_label, dplyr::all_of(rest_cols)) %>%
    tidyr::pivot_longer(cols = -period_label, names_to = "var", values_to = "value") %>%
    dplyr::mutate(value_num = to_num(value)) %>%
    dplyr::group_by(var) %>%
    dplyr::mutate(min_v = min(value_num, na.rm=TRUE),
                  max_v = max(value_num, na.rm=TRUE),
                  scaled = ifelse(is.finite(min_v) & is.finite(max_v) & (max_v - min_v) > 0,
                                  (value_num - min_v) / (max_v - min_v),
                                  NA_real_)) %>%
    dplyr::ungroup()

  p_rest_scaled <- ggplot2::ggplot(rest_long, ggplot2::aes(x=period_label, y=scaled, group=va
    ggplot2::geom_line(size=1) +
    ggplot2::geom_point(size=1.6) +
    ggplot2::labs(title="Other variables (min-max scaled) across biweekly periods", x="Biweel
    ggplot2::theme_minimal() +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 45, hjust = 1), plot.title = g
  print(p_rest_scaled)

  # small multiples: raw values
```

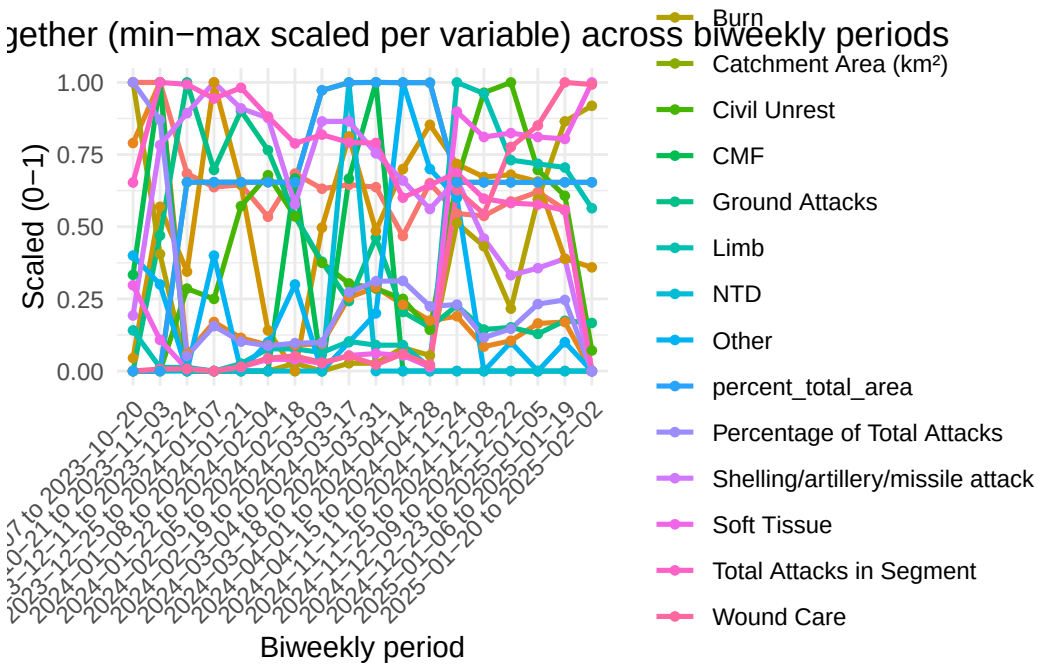
```
  rest_long_raw <- rest_long %>% dplyr::mutate(value_num = as.numeric(value_num))
  p_rest_facets <- ggplot2::ggplot(rest_long_raw, ggplot2::aes(x=period_label, y=value_num, g
    ggplot2::geom_line() + ggplot2::geom_point(size=1) +
    ggplot2::facet_wrap(~var, scales="free_y", ncol=2) +
    ggplot2::labs(title="Other variables (raw values) - one panel per variable", x="Biweekly
    ggplot2::theme_minimal() +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 45, hjust = 1), plot.title = g
  print(p_rest_facets)

} else {
  plot.new(); text(0.5,0.5,"No 'rest' variables found to plot")
}
```






```
# --- 4) Big plot: all variables together (min-max scaled) across categorical x-axis ---
all_plot_cols <- unique(c(attack_cols, injury_cols, rest_cols))
all_plot_cols <- all_plot_cols[!is.na(all_plot_cols)]
if (length(all_plot_cols) > 0) {
  all_long <- df %>%
    dplyr::select(period_label, dplyr::all_of(all_plot_cols)) %>%
    tidyr::pivot_longer(cols = -period_label, names_to = "var", values_to = "value") %>%
    dplyr::mutate(value_num = to_num(value)) %>%
    dplyr::group_by(var) %>%
    dplyr::mutate(min_v = min(value_num, na.rm=TRUE),
                  max_v = max(value_num, na.rm=TRUE),
                  scaled = ifelse(is.finite(min_v) & is.finite(max_v) & (max_v - min_v) > 0,
                                  (value_num - min_v) / (max_v - min_v),
                                  NA_real_)) %>%
    dplyr::ungroup()

  p_all <- ggplot2::ggplot(all_long, ggplot2::aes(x=period_label, y=scaled, group=var, color=
    ggplot2::geom_line(size=0.9) +
    ggplot2::geom_point(size=1.2) +
    ggplot2::labs(title="All variables together (min-max scaled per variable) across biweekly
    ggplot2::theme_minimal() +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 45, hjust = 1), plot.title = g
  print(p_all)
```

```
} else {
  plot.new(); text(0.5,0.5,"No variables found to include in the 'all variables' plot")
}
```




```
# --- End of chunk ---
# Tips:
# - To reduce x-axis label crowding, show only every N-th label:
#     + Add: + scale_x_discrete(breaks = levels(df$period_label)[seq(1, nlevels(df$period_lab
# - To change scaling to z-scores:
#     replace the 'scaled' calculation with:
#       scaled = (value_num - mean(value_num, na.rm=TRUE)) / sd(value_num, na.rm=TRUE)
# - If you want lines to *not* connect across calendar gaps (when using date x rather than ca
#   I can add logic to create break groups. With the categorical period_label approach (used
#   spacing is even and lines connect only adjacent periods.
```

```
#totaldata |>
  #filter(`Hospital` == "Al Shifa Medical Hospital") |>


# Namespace-safe chunk: group injuries by hospital (Al Shifa, European, Nasser) and plot dist
# Requires: readxl, dplyr, tidyr, ggplot2, stringr
# Install if needed: install.packages(c("readxl","dplyr","tidyr","ggplot2","stringr"))
```

```r
# Load a plotting library (we'll fully qualify other functions)
suppressMessages({ library(ggplot2) })

# --- Read file ---
xlsx_candidates <- c("aggregated_dataset.xlsx", "/mnt/data/aggregated_dataset.xlsx")
dataset_path <- NULL
for (p in xlsx_candidates) if (file.exists(p)) { dataset_path <- p; break }
if (is.null(dataset_path)) stop("Could not find aggregated_dataset.xlsx. Place it in the work

df <- readxl::read_xlsx(dataset_path)

# --- Normalize column names ---
names(df) <- stringr::str_trim(names(df))
names(df) <- stringr::str_replace_all(names(df), "[\\s]+", " ")  # collapse multi-spaces
lc_names <- tolower(names(df))

# --- 1) Find a hospital column (try common names) ---
hospital_candidates <- c("hospital", "hospital_name", "facility", "facility_name", "site", "s
hospital_col <- NA_character_
for (cand in hospital_candidates) {
  i <- which(lc_names == tolower(cand))
  if (length(i)>0) { hospital_col <- names(df)[i[1]]; break }
}
# fuzzy contains if exact not found
if (is.na(hospital_col)) {
  for (cand in hospital_candidates) {
    i <- which(stringr::str_detect(lc_names, stringr::str_replace_all(tolower(cand), "[^a-z0-
    if (length(i)>0) { hospital_col <- names(df)[i[1]]; break }
  }
}

if (is.na(hospital_col)) {
  warning("No hospital/facility column detected. I will attempt to proceed by looking for hos
}

# --- 2) Define the injury type names we expect (user-provided list) ---
injury_vars_user <- c("Burn","CMF","Limb","NTD","Soft Tissue","Wound Care")

# fuzzy matcher to find actual injury columns in df
fuzzy_match_cols <- function(user_vec, available_names) {
  avail_lc <- tolower(available_names)
  sapply(user_vec, function(u){
```

```
    u_lc <- tolower(u)
    # exact
    idx <- which(avail_lc == u_lc); if (length(idx)>0) return(available_names[idx[1]])
    # contains
    idx <- which(stringr::str_detect(avail_lc, stringr::fixed(u_lc))); if (length(idx)>0) re
    # normalized (remove non-alphanum)
    u_norm <- stringr::str_replace_all(u_lc, "[^a-z0-9]+", "")
    idx <- which(stringr::str_replace_all(avail_lc, "[^a-z0-9]+", "") == u_norm); if (length
    # partial word-match
    idx <- which(stringr::str_detect(avail_lc, stringr::str_replace_all(u_lc, "[^a-z0-9]+",
    NA_character_
  }, USE.NAMES = FALSE)
}

injury_cols <- fuzzy_match_cols(injury_vars_user, names(df))
injury_map <- data.frame(requested = injury_vars_user, matched = injury_cols, stringsAsFacto
# drop NA matches
injury_cols_clean <- unique(na.omit(injury_cols))

if (length(injury_cols_clean) == 0) stop("No injury columns (Burn/CMF/Limb/NTD/Soft Tissue/W

# Show mapping for user's info
cat("Injury columns matched:\n")
```

```
Injury columns matched:
```

```
print(injury_map)
```

```
    requested     matched
1        Burn        Burn
2         CMF         CMF
3        Limb        Limb
4         NTD         NTD
5 Soft Tissue Soft Tissue
6  Wound Care  Wound Care
```

```
# --- 3) Identify rows for each hospital (case-insensitive contains) ---
# We'll look for these three hospital name fragments:
hospital_targets <- c("al shifa", "european", "nasser")
```

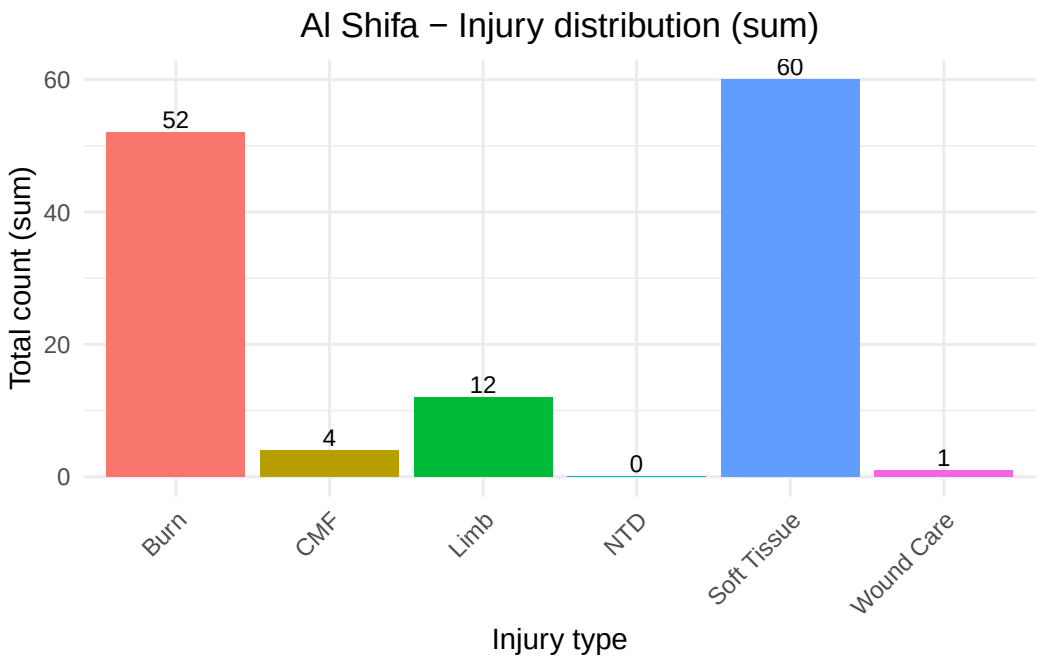
```r
# Helper to find values in hospital_col OR across all character columns if hospital_col missi
find_rows_for_hospital <- function(df, hosp_pattern) {
  if (!is.na(hospital_col)) {
    vals <- as.character(df[[hospital_col]])
    sel <- stringr::str_detect(tolower(vals), stringr::fixed(hosp_pattern))
    return(which(sel))
  } else {
    # search across all character columns
    char_cols <- names(df)[sapply(df, function(x) is.character(x) || is.factor(x))]
    if (length(char_cols)==0) return(integer(0))
    sel_any <- rep(FALSE, nrow(df))
    for (c in char_cols) {
      sel_any <- sel_any | stringr::str_detect(tolower(as.character(df[[c]])), stringr::fixe
    }
    return(which(sel_any))
  }
}

# Collect the indices for each hospital
hospital_indices <- lapply(hospital_targets, function(pat) find_rows_for_hospital(df, pat))
names(hospital_indices) <- hospital_targets

# Diagnostics: any not found?
for (i in seq_along(hospital_targets)) {
  target <- hospital_targets[i]
  idx <- hospital_indices[[i]]
  if (length(idx)==0) message(sprintf("Warning: No rows found for hospital matching '%s'.", t
  else message(sprintf("Found %d rows for hospital matching '%s'.", length(idx), target))
}
```

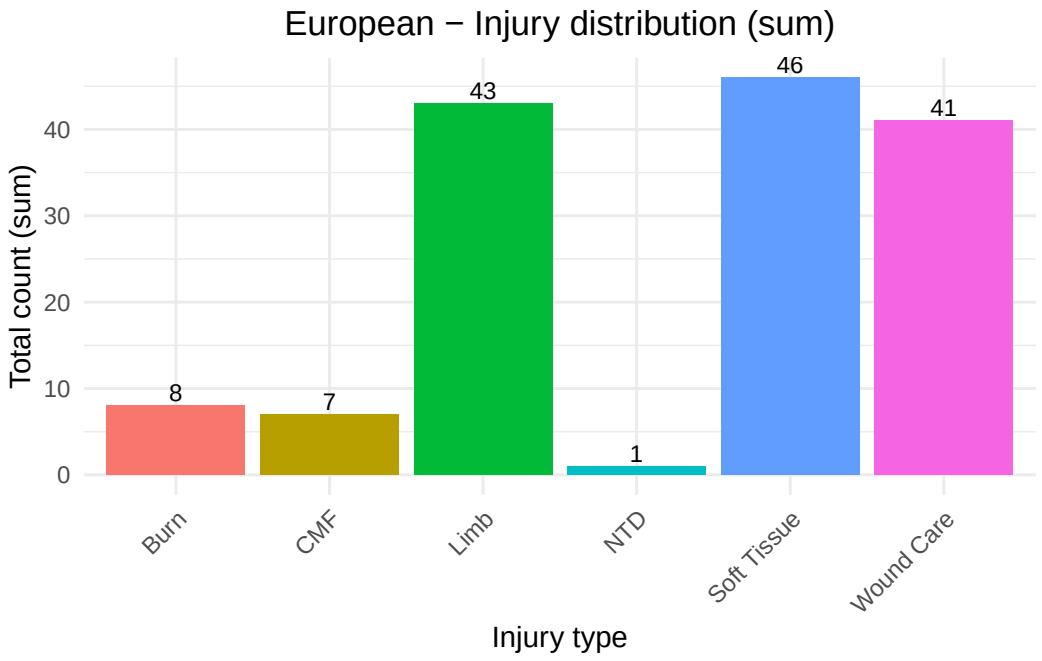
```
Found 2 rows for hospital matching 'al shifa'.

Found 10 rows for hospital matching 'european'.

Found 6 rows for hospital matching 'nasser'.
```

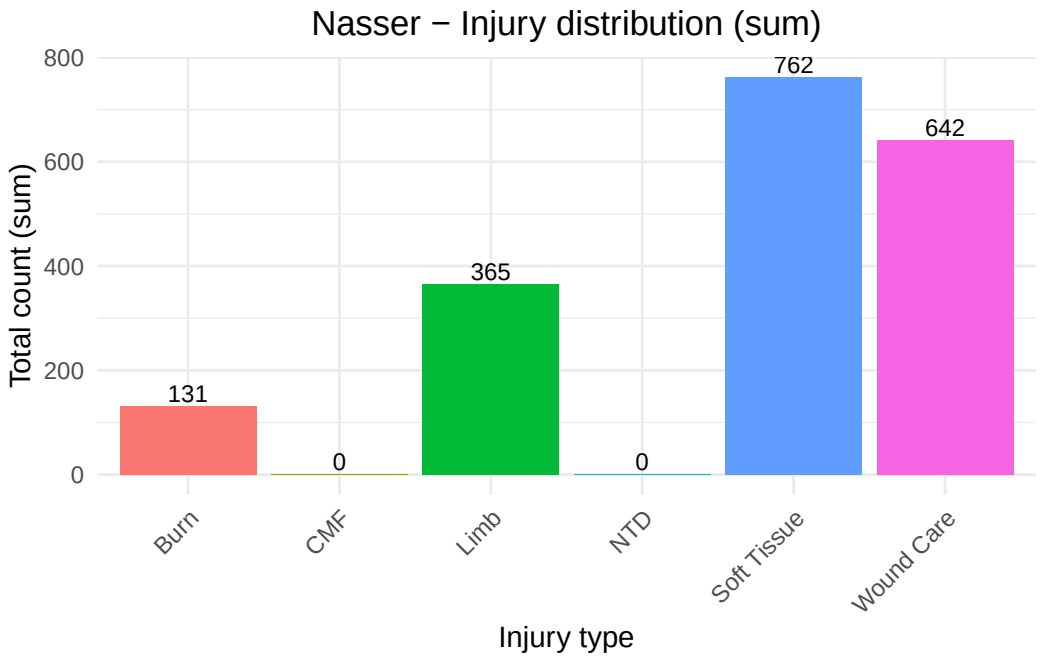
```r
# --- 4) For each hospital, sum the injury counts across rows and prepare a tidy table ---
to_num <- function(x) suppressWarnings(as.numeric(x))

hospital_summaries <- list()
for (i in seq_along(hospital_targets)) {
```

```
  target <- hospital_targets[i]
  idx <- hospital_indices[[i]]
  if (length(idx)==0) {
    hospital_summaries[[target]] <- NULL
    next
  }
  # subset rows and select the matched injury columns
  sub <- df[idx, , drop = FALSE]
  # ensure columns exist in sub
  present_inj_cols <- intersect(injury_cols_clean, names(sub))
  if (length(present_inj_cols)==0) {
    hospital_summaries[[target]] <- NULL
    next
  }
  # coerce to numeric & sum
  sums <- sapply(present_inj_cols, function(col) sum(to_num(sub[[col]]), na.rm = TRUE))
  sums_df <- data.frame(hospital = target,
                        injury_type = names(sums),
                        count = as.numeric(sums),
                        stringsAsFactors = FALSE)
  # Make injury_type labels prettier (use original requested name when matched)
  # Map back to requested if possible:
  sums_df$injury_label <- sapply(sums_df$injury_type, function(matched) {
    # find first requested whose matched equals this matched
    req_idx <- which(injury_map$matched == matched)
    if (length(req_idx)>0) injury_map$requested[req_idx[1]] else matched
  })
  hospital_summaries[[target]] <- sums_df
}

# Combine summaries into one data.frame
summary_combined <- do.call(rbind, hospital_summaries)
if (is.null(summary_combined) || nrow(summary_combined)==0) stop("No hospital injury summarie

# Reorder injury_label factor to keep consistent ordering
summary_combined$injury_label <- factor(summary_combined$injury_label, levels = unique(summar

# --- 5) Plot: one bar plot per hospital (printed sequentially) ---
unique_hospitals_found <- unique(summary_combined$hospital)

for (h in unique_hospitals_found) {
  d_h <- summary_combined[summary_combined$hospital == h, , drop = FALSE]
```

```
  # use requested hospital display name (title-case)
  display_name <- stringr::str_to_title(h)
  p <- ggplot2::ggplot(d_h, ggplot2::aes(x = injury_label, y = count, fill = injury_label)) +
    ggplot2::geom_col(show.legend = FALSE) +
    ggplot2::geom_text(ggplot2::aes(label = count), vjust = -0.25, size = 3) +
    ggplot2::labs(title = paste0(display_name, " - Injury distribution (sum)"),
                  x = "Injury type", y = "Total count (sum)") +
    ggplot2::theme_minimal() +
    ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 45, hjust = 1),
                   plot.title = ggplot2::element_text(hjust = 0.5))
  print(p)
}
```

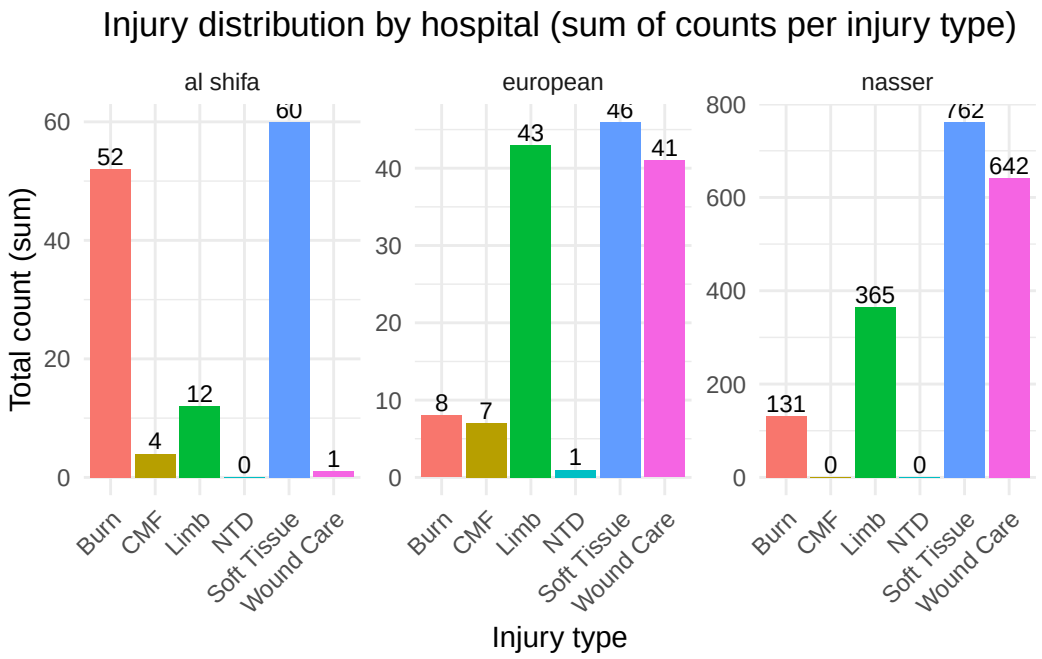







```
# --- 6) Combined faceted bar plot (3 panels) for comparison ---
p_combined <- ggplot2::ggplot(summary_combined, ggplot2::aes(x = injury_label, y = count, fi
  ggplot2::geom_col(show.legend = FALSE) +
  ggplot2::geom_text(ggplot2::aes(label = count), vjust = -0.25, size = 3) +
```

```
  ggplot2::facet_wrap(~ hospital, scales = "free_y") +
  ggplot2::labs(title = "Injury distribution by hospital (sum of counts per injury type)",
                x = "Injury type", y = "Total count (sum)") +
  ggplot2::theme_minimal() +
  ggplot2::theme(axis.text.x = ggplot2::element_text(angle = 45, hjust = 1),
                 plot.title = ggplot2::element_text(hjust = 0.5))

print(p_combined)
```




```
# --- End ---
# Notes:
# - If any hospital returned "no rows found" or had zero sums, check your hospital column and
# - If you'd like the hospital display names to be exactly "Al Shifa Medical Hospital", "Euro
# - Want the counts normalized (percent of hospital) instead of raw sums? I can add that.
```